QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

Clinical Diagnostics Triage & Workflow Copilot

Team Shrooms
danielzfchan@gmail.com

Document Control

Field	Detail
System Under Test (SUT)	Clinical Diagnostics Triage & Workflow Copilot — Team Shrooms
Team Repository URL	https://github.com/[team-repo]
Project Board URL	N/A — internal task tracking used for hackathon sprint
Live Deployment URL	N/A — Windows desktop application, no hosted deployment
Version	1.0 — Preliminary Round

Objective: The primary objective of this testing documentation is to ensure that the Clinical Diagnostics Triage & Workflow Copilot reliably performs clinical note intake, GLM-assisted OCR for handwritten notes, local AI model inference for ECG signals and chest X-ray images, triage synthesis with confidence-based human-review routing, and mock FHIR R4 ClinicalImpression output generation, across all intended inputs and failure conditions, with sufficient traceability for demonstration and evaluation.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns testing back to specific system requirements through a Requirements Traceability Matrix. It ensures that every in-scope feature has a corresponding test coverage plan, and that out-of-scope items are explicitly excluded to prevent unplanned feature creep during the testing phase.

1.1 In-Scope Core Features

- **Clinical Note Intake:** Typed note submission via WinForms, note passed to Python engine as command-line argument, session title generation from first 35 characters.
- **GLM-Assisted OCR:** Handwritten note image upload to Z.AI file API, ilmu-glm-5.1 invocation with retrieval tool and OCR system prompt, unreadable-image sentinel detection and human-review routing.
- **Biomedical File Attachment:** Single file attachment per session via WinForms file picker, extension filtering to .csv, .tsv, .json, .png, .jpg, and .jpeg, file path forwarding to Python engine.
- **GLM-Driven Model Routing (GLM Call 1):** Structured JSON routing decision from the first GLM call, correct identification of requires_model flag and AI_nodes list, model name and file path resolution.
- **ECG Signal Classification:** Heartbeat_Abnormality_Model inference on .csv and .tsv ECG files, five-class output with confidence score, preprocessing variant support (forward-fill, zero-fill, drop-missing).
- **Chest X-Ray Classification:** Chest_XRay_Vision_Model inference on .png, .jpg, and .jpeg image files, binary Normal or Pneumonia output with confidence score.
- **NLP Summarization Fallback:** Token estimation before first GLM call, Clinical_Note_Summarization_Model invocation when ceiling is exceeded, summarized note replacement for all downstream calls.
- **GLM-Driven Triage Synthesis (GLM Call 2):** Structured triage JSON output with triage_priority (1–5), clinical_summary, recommended_action, triage_reason, and flagged_for_human_review, 90 percent confidence threshold enforcement, contradiction detection and routing.

- **Mock FHIR R4 ClinicalImpression Output:** FHIR_R4_Handling.py mapping of triage JSON to ClinicalImpression resource with resourceType, status, subject, description, summary, finding, investigation, and two custom extensions.
- **Human-Review Routing & WinForms Warning:** Red warning banner display on flagged_for_human_review true, triage_priority 5 assignment for all non-confident or failure-adjacent sessions.
- **Session Activity Logging:** activity_logger() write to date-stamped TSV under Activity_Log/ for every API call, OCR execution, and model node execution, including error events before RuntimeError is raised.

1.2 Out-of-Scope

- Multi-file attachment from the WinForms desktop UI (Python engine supports it architecturally but the interface does not expose it).
- FHIR ClinicalImpression display in the WinForms result panel (current output is stdout only).
- API timeout enforcement and bounded retry logic on GLM calls.
- Configurable Python executable path (currently hardcoded to development workstation).
- Live write-back to a certified FHIR server or production EHR system.
- Mobile or web-based client interface.
- Multi-user concurrent sessions.
- DICOM-native imaging metadata preservation.

2. Risk Assessment & Mitigation Strategy

Quality assurance risks are anticipatory. The table below identifies technical risks specific to the system's architecture, evaluates each using a 5x5 Risk Assessment Matrix, and defines a mitigation strategy and testing approach for each. Risk Score = Likelihood × Severity.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach

GLM routing agent returns incorrect file path in AI_nodes response	3	4	12 (High)	Pass the attached file path explicitly through the note context so the routing agent can reference it directly rather than inferring it. Validate the returned file_path field against the actual attachment path before invoking local model inference.	Submit a note with an file and verify tha AI_nodes file_path i routing JSON matche actual argument pass the C# process b Force a mismatch confirm that the local r raises a file-not-found that is logged and surf
OCR extracts partially correct text from a low-legibility handwritten note without triggering the sentinel	3	4	12 (High)	Add a minimum word count check and basic coherence validation on OCR output before passing to the routing GLM. Flag sessions where OCR output is below a word threshold as requiring human review.	Submit a delibe degraded handwritten image. Verify the output. Check whethe triage synthesis performed on the deg text or whether a hu review flag is raised the activity record compare detail content.
GLM returns valid prose but not a JSON object, causing silent downstream parse failure	2	5	10 (Medium)	Both GLM calls strip markdown fences and attempt JSON parsing with a try-except block. Add a retry with a stricter output-format reminder before final failure. Surface the JSON parse error explicitly in the WinForms panel	Inject a malforme prose-only GLM res by mocking the AP return value. Confirm the exception is ca logged to Activity TSV with the correct type, and surfaced System Error in WinForms chat

				rather than a generic System Error.	without crashing process.
Confidence threshold enforced only by GLM prompt instruction, not by independent numeric check	3	4	12 (High)	Add a programmatic numeric confidence check in the Python engine on each AI node result before passing to the synthesis call. If any node confidence is below 0.90, override the synthesis prompt to set flagged_for_human_review to true regardless of model response.	Submit an ECG file v deliberately confidence pred (below 0.90 thres Verify flagged_for_human_r is true and triage_prio 5 in the output. Confir WinForms red wa banner is displayed.
Hardcoded Python executable path causes silent process invocation failure on any non-development machine	5	5	25 (Critical)	Replace the hardcoded path in Form1.cs with a PATH-based python lookup or a configurable settings file before the demonstration build. Test on a second machine before any live demonstration.	Run the WinF application on a ma that does not have Pyth the hardcoded path. V that a meaningful message is displayed than a silent hang. PATH-based resoluti the replacement.
No API timeout enforcement causes indefinite UI hang on slow or stalled GLM response	3	4	12 (High)	Add a timeout parameter to both GLM API calls before the demonstration build. Surface a timeout error in the WinForms UI and write the timeout event to the Activity_Log TSV.	Simulate a stalled response by introduci artificial delay in a m API call. Verify tha WinForms UI does hang indefinitely and timeout error is disp

					after the confi threshold.
Token estimation formula underestimates actual token cost for certain file types, causing an unexpected API-level token limit error	2	3	6 (Medium)	Use a conservative estimation multiplier or integrate a tokenizer library for precise counts before the first GLM call. Add a buffer margin below the 200,000-token ceiling to absorb estimation variance.	Submit progressively note and file combina and monitor whethe token limit is trigger the API level befor local summariz fallback fires. Verify API-level TokenLimit events are caught, lo and surfaced correctly

Risk Assessment Scoring Criteria:

Likelihood	Description	Severity	Description
1	Rare	1	Negligible impact
2	Unlikely	2	Minor impact
3	Possible	3	Moderate impact
4	Likely	4	Major impact
5	Almost Certain	5	Critical system failure

Risk Score = Likelihood × Severity

Risk Score	Risk Level	Recommended Action
1 – 5	Low	Monitor only. Acceptable for demonstration scope.
6 – 10	Medium	Mitigate before demonstration. Include in test plan.
11 – 15	High	Must mitigate. Thorough testing required.
16 – 25	Critical	Highest priority. Extensive testing and blocking fix required.

3. Test Environment & Execution Strategy

This section describes where testing takes place, how test data is managed, and the rules governing each testing phase. All testing for this system is performed locally on a Windows workstation due to the desktop-only architecture and the absence of a hosted deployment or CI/CD pipeline in the current build.

- **Unit Testing:**
  - Scope: Individual Python functions in Model_Usage.py and FHIR_R4_Handling.py, including token_estimator(), activity_logger(), FHIR_R4_ClinicalImpression_Resource_Handler(), and each model inference wrapper (Heartbeat_Abnormality_Model, Chest_XRay_Vision_Model, Clinical_Note_Summarization_Model).

- Execution: Manual invocation of each function with prepared input fixtures. External dependencies including the Z.AI GLM API are mocked to isolate function logic.
- Isolation: All GLM API calls are replaced with mock return objects that simulate valid JSON, malformed JSON, TokenLimitError, and general Exception responses. File system reads are performed on local fixture files.
- Pass Condition: Each function returns the expected output type and value for happy inputs, raises the expected exception type with the correct message for failure inputs, and writes the correct record to the Activity_Log TSV.

- **Integration Testing:**
  - Scope: End-to-end flow from the C# WinForms process invocation through the Python engine and back to the WinForms result panel, covering the full argument passing, JSON payload exchange, and UI result rendering.
  - Execution: Performed after unit testing passes. A complete session is submitted through the WinForms interface with a typed note and each supported file type individually.
  - Workflow: Real GLM API calls are made. Real local model inference is performed. No mocking is applied at the integration layer.
  - Pass Condition: The WinForms result panel displays the correct triage priority, clinical summary, recommended action, and triage reason. The Activity_Log TSV contains a complete record of all events from the session. The FHIR output is printed to stdout with all required fields populated.

- **Test Environment:**
  - Local Testing: All testing is performed manually on the development workstation running Windows with the Python 3.12 runtime, TensorFlow, Keras, and the zai client library installed. The WinForms application is built and run from Visual Studio.
  - No CI/CD Pipeline: The current build does not include a continuous integration pipeline. All test execution is triggered manually by the developer or QA team member.

- Model Files: All model weight files must be present under Model_Usage/Models/ and the directory structure must match the path resolution logic in the WinForms application before any test session is executed.

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase: Regression testing is run across all in-scope features each time a significant change is made to Model_Usage.py, FHIR_R4_Handling.py, or Form1.cs.
  - Pass/Fail Condition: A test case passes only when the actual outcome exactly matches the expected outcome defined in the test case specification. Any mismatch, missing field, incorrect routing decision, or absent log entry is recorded as a failure and added to the defect log.
  - Continuation Rule: Unit test cases for a given module must achieve the defined pass rate threshold before integration testing proceeds for that module. Regression testing does not proceed if any critical test case is in a failed state.

- **Test Data Strategy:**
  - Manual: A set of five prepared test inputs is used, covering a typed clinical note only, a typed note with ECG CSV file, a typed note with chest X-ray image, a handwritten note image, and a note designed to exceed the 200,000-token context window estimate.
  - Edge Case Fixtures: Prepared inputs include an unsupported file extension, an empty note submission, a corrupted image file, a low-legibility handwritten note scan, and a GLM mock return with a non-JSON prose response.
  - Mock Patient Data: All patient identifiers, practitioner references, and FHIR subject fields use synthetic placeholder values. No real patient data is used in any test session.

- **Passing Rate Threshold:**
  - A minimum of 85 percent of all executed test cases must pass for the test phase to be considered complete. For critical test cases (confidence threshold routing, human-review flag display, error logging, and process bridge stability), a 100 percent pass rate is required with no exceptions.

4. Release Thresholds & Demonstration Gates

This section defines the success metrics and thresholds that must be met before the system is considered ready for live demonstration. Because the system uses a local desktop architecture with no CI/CD pipeline, these thresholds function as manual gate checks performed by the QA team before the demonstration session.

4.1 Integration Thresholds (Pre-Demonstration Build)

These checks must pass before the demonstration build is packaged and deployed to the demonstration workstation.

Check	Requirement	Current Status	Pass / Fail
Process Bridge Stability	Zero unhandled process crashes across 10 consecutive sessions	Pending	Pending
Unit Test Pass Rate	100% passing on all critical path functions	Pending	Pending
Python Path Resolution	Successful invocation on at least two distinct machines	Pending	Pending
Activity Log Write Verification	Every test session produces a correctly structured TSV entry for each event	Pending	Pending
Error Surface Verification	All exception paths display a System Error in WinForms	Pending	Pending

	without process crash		

4.2 Demonstration Thresholds (Live Session Readiness)

These checks must pass before any live demonstration session is conducted.

Check	Requirement	Current Status	Pass / Fail
End-to-End Regression Pass Rate	Minimum 85% of all test cases passing	Pending	Pending
Critical Test Case Pass Rate	100% — routing, threshold, logging, error surface	Pending	Pending
GLM API Response Time	Average below 8,000ms per call under standard conditions	Pending	Pending
Local Model Inference Time	Average below 3,000ms per model invocation on the demonstration machine	Pending	Pending
AI Output Pass Rate	Minimum 80% of documented prompt-response test pairs produce an acceptable output as defined in Section 6	Pending	Pending

API Key Exposure Check	Zero API keys present in source code, logs, or stdout output	Pending	Pending
Zero Critical Bugs	No P0 or P1 defects open at demonstration time	Pending	Pending

5. Test Case Specifications (Drafts)

This section documents test cases covering the complete happy path, targeted negative and edge cases, and non-functional performance tests. Each test case is mapped to a specific in-scope feature and includes test steps, expected results, and an actual result field to be completed during execution.

Test Case ID	Type & Mapped Feature	Test Description	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow):** Typed Note with ECG File	Verify that a physician can submit a typed clinical note with an ECG CSV file and receive a complete structured triage output with FHIR R4 ClinicalImpression printed to stdout and the result displayed in the WinForms panel.	**Steps:** 1. Launch WinForms. 2. Type a clinical note describing cardiac symptoms. 3. Attach a valid ECG .csv file. 4. Click Send. 5. Observe WinForms	**Expected:** Triage priority 1–4 displayed in WinForms. clinical_summary, recommended_action, and triage_reason populated. flagged_for_human_review is false. FHIR ClinicalImpression printed to stdout with all required fields. Activity_Log TSV contains Model_Choosing and Final_Analysis events. Session completes without error.

			result panel and stdout.	
TC-02	**Happy Case (Entire Flow):** Typed Note with Chest X-Ray Image	Verify that a physician can submit a typed clinical note with a chest X-ray image and receive a complete triage output with Normal or Pneumonia classification incorporated into the synthesis result.	**Steps:** 1. Launch WinForms. 2. Type a note describing respiratory symptoms. 3. Attach a valid chest X-ray .png file. 4. Click Send. 5. Observe result panel and stdout.	**Expected:** Chest_XRay_Vision_Model result appears in triage_reason. FHIR investigation block includes the imaging finding. All other fields as TC-01. Activity_Log contains a model node execution entry with the correct event type.
TC-03	**Happy Case:** Handwritten Note OCR Flow	Verify that a handwritten clinical note image is successfully converted to machine-readable text by the GLM OCR step and that the extracted text is used as the input for triage synthesis.	**Steps:** 1. Launch WinForms. 2. Attach a clear handwritten note image. 3. Click Send with no additional file. 4. Observe result panel.	**Expected:** OCR GLM call completes without raising a RuntimeError. Extracted text appears as the note context in the triage synthesis prompt. Triage result is produced based on the handwritten note content. Activity_Log contains an OCR event entry.
TC-04	**Negative Case:** Empty Note Submission	Verify that submitting a session with no typed note and no file attachment is	**Steps:** 1. Launch WinForms. 2.	**Expected:** Send button triggers a warning dialog indicating that a clinical note

		blocked by the WinForms interface before the Python engine is invoked.	Leave the note input field empty. 3. Do not attach any file. 4. Click Send.	is required. No process invocation occurs. No Activity_Log entry is written. Application remains stable.
TC-05	**Negative Case:** Unsupported File Extension	Verify that attaching a file with an unsupported extension (e.g. .docx or .mp4) is rejected by the Python engine with a controlled error and does not crash the application.	**Steps:** 1. Launch WinForms. 2. Type a clinical note. 3. Attempt to attach a .docx file. 4. Click Send.	**Expected:** The file picker either filters out the unsupported extension, or the Python engine raises a ValueError and surfaces a System Error in the WinForms panel. Application remains stable. Error is logged to Activity_Log TSV.
TC-06	**Negative Case (Edge):** Below-Threshold Model Confidence	Verify that a session where the local model returns a confidence score below 90 percent results in flagged_for_human_review being set to true and triage_priority being set to 5, with the WinForms red warning banner displayed.	**Steps:** 1. Prepare a fixture ECG input designed to produce a borderline or ambiguous classification. 2. Submit via WinForms. 3. Observe triage output fields and WinForms panel.	**Expected:** flagged_for_human_review is true in triage JSON. triage_priority is 5. WinForms displays the red HUMAN INTERVENTION REQUIRED banner. FHIR status is in-progress. Activity_Log records the session with all events.

TC-07	**Negative Case:** GLM API Call Failure	Verify that a GLM API failure during the model routing or triage synthesis stage is caught, logged, and surfaced as a System Error in the WinForms panel without crashing the process.	**Steps:** 1. Mock or simulate a GLM API error (e.g. invalid API key, network block). 2. Submit a valid note through WinForms. 3. Observe WinForms panel and Activity_Log.	**Expected:** A System Error message is displayed in the WinForms chat panel. The Activity_Log TSV contains an entry with the event type (Model_Choosing_API_Error or Final_Analysis_API_Error) and the full exception detail. The process does not crash.
TC-08	**NFR (Performance):** GLM API Response Time	Verify that each GLM API call completes within the acceptable response time threshold under standard network conditions.	**Steps:** 1. Submit 10 consecutive sessions with the same typed note and no file attachment. 2. Record the elapsed time from process invocation to Python engine completion for each	**Expected:** Average GLM API response time below 8,000ms per call. No session exceeds 15,000ms. Zero API error rate across the 10 sessions.

			session. 3. Calculate the average GLM call duration from the Activity_Log TSV timestamps.	
TC-09	**NFR (Performance):** Local Model Inference Time	Verify that the local TensorFlow and Keras model inference completes within the acceptable time threshold for both ECG and chest X-ray inputs.	**Steps:** 1. Submit 5 sessions with an ECG CSV file. Record inference duration from Activity_Log. 2. Submit 5 sessions with a chest X-ray image. Record inference duration from Activity_Log. 3. Calculate the average for each model.	**Expected:** Average inference time below 3,000ms per model invocation for both ECG and chest X-ray on the demonstration machine. No session inference exceeds 6,000ms.

6. AI Output & Boundary Testing (Drafts)

This section validates that GLM integration produces acceptable structured output and handles abnormal inputs gracefully. All acceptance criteria are defined against the JSON schema requirements enforced in the system prompt of each GLM call.

6.1 Prompt / Response Test Pairs

The following test pairs document the required inputs, acceptance criteria, and actual results for the core GLM interactions in the system. A passing response must satisfy all acceptance criteria listed; partial satisfaction is recorded as a failure.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	**GLM Call 1 (Model Routing):** A typed note describing chest pain and shortness of breath submitted alongside an ECG CSV file path.	The response must be a valid JSON object containing requires_model: true, and an AI_nodes array with exactly one entry specifying model_name: 'Heartbeat_Abnormality_Model', a non-empty file_path, an empty or null modifications field, and a headers array. No prose, no markdown fences, no additional keys outside the schema.	To be completed during test execution.	Pending
AI-02	**GLM Call 2 (Triage Synthesis with ECG Result):** A typed note describing chest pain combined with a Heartbeat_Abnormality_Model result of 'Ventricular Ectopic Beats' at 0.94 confidence.	The response must be a valid JSON object containing triage_priority as an integer between 1 and 4, a non-empty clinical_summary string, a non-empty recommended_action string, a non-empty triage_reason string that references the ECG finding, and flagged_for_human_review: false. No hallucinated patient	To be completed during test execution.	Pending

		identifiers. No fabricated model names.		
AI-03	**GLM Call 2 (Triage Synthesis without Model):** A typed note describing mild headache and fatigue with no biomedical file attached. requires_model is false.	The response must be a valid JSON object with triage_priority between 1 and 4, all required text fields non-empty, and flagged_for_human_review: false. The triage_reason must not reference any AI model finding or fabricate a model result that was not produced.	To be completed during test execution.	Pending
AI-04	**GLM OCR Call:** A clear handwritten clinical note image of approximately 80 words describing presenting symptoms, duration, and observations, uploaded via the Z.AI file API.	The response must be a plain text string containing at least 60 readable words that accurately reflect the handwritten content. The sentinel string <<flagged_for_human_review>> must not appear. No fabricated clinical content not present in the original image.	To be completed during test execution.	Pending
AI-05	**GLM Call 2 (Contradiction Scenario):** A typed note stating the patient shows no cardiac abnormality combined with a Heartbeat_Abnormality_Model result of 'Ventricular Ectopic Beats' at 0.96 confidence.	The response must set flagged_for_human_review: true and triage_priority: 5. The triage_reason must explicitly reference the contradiction between the note content and the model finding. No confident triage action may be recommended.	To be completed during test execution.	Pending

6.2 Oversized / Large Input Test

This section defines the maximum input size the system accepts and documents what occurs when the limit is exceeded.

Field	Detail
Maximum Input Size	200,000 tokens (combined estimate of system prompt, user note, and attached file content using character count divided by four for text and file size divided by four for document-type files).
Input Used While Testing	A synthetic clinical note of approximately 180,000 estimated tokens combined with a large .txt file attachment estimated at 30,000 tokens, totalling approximately 210,000 tokens — approximately 1.05x the ceiling.
Expected Behavior	The token_estimator() function detects that the combined estimate exceeds the 200,000-token ceiling and invokes the Clinical_Note_Summarization_Model before the first GLM call. The summarized note (maximum 200 tokens) replaces the full note for all downstream processing. The session completes without a TokenLimitError from the API.
Actual Behavior	To be completed during test execution.
Status	Pending

6.3 Adversarial / Edge Prompt Test

The following test documents how the system handles an ambiguous or malformed prompt that does not clearly indicate a diagnostic modality and cannot be resolved to a specific model routing decision.

Field	Detail

Adversarial Input	A note containing only: 'Patient came in. Please check everything.' with no attached biomedical file.
Expected System Behavior	The routing GLM call returns requires_model: false and an empty AI_nodes array because no modality can be identified. The synthesis GLM call produces a triage output based on note content alone, sets triage_priority to a conservative value (3 or higher), and sets flagged_for_human_review: true due to insufficient clinical detail. The session does not crash.
Acceptance Criteria	The system completes the session without raising an unhandled exception. The triage JSON is structurally valid. flagged_for_human_review is true. The triage_reason explicitly references insufficient clinical information as the basis for routing to human review.
Actual Behavior	To be completed during test execution.
Status	Pending

6.4 Hallucination Handling

This section describes the mechanisms the system uses to detect or mitigate hallucinated responses from the GLM at each integration point.

- **Schema-Enforced Output Validation:** Both GLM calls define a strict JSON schema in the system prompt and require the model to return only a valid JSON object matching that schema. Any response that cannot be parsed as JSON after stripping markdown fences is treated as a failure, logged, and surfaced as a System Error. This prevents hallucinated prose from being silently accepted as a structured triage decision.
- **Confidence Threshold as a Hallucination Boundary:** The synthesis GLM prompt instructs the model to set flagged_for_human_review to true and triage_priority to 5 whenever any model confidence is below 90 percent or a contradiction exists between note content and model output. This prevents the model from generating a confident triage recommendation when the underlying evidence is ambiguous or contradictory.

- **OCR Sentinel Detection:** The OCR GLM call checks for the exact return string <<flagged_for_human_review>> as a signal that the model cannot extract readable text. This provides a defined fallback channel that is not dependent on the model generating a structurally well-formed response when the input is unreadable.
- **No Fabricated Model Results:** The synthesis prompt is conditionally constructed at runtime. If requires_model is false and no AI node results are present, the prompt explicitly instructs the model not to reference or fabricate any model finding. This prevents the synthesis stage from hallucinating a diagnostic result that was never produced by local inference.
- **Activity Log as Audit Trail:** Every GLM response is written in full to the Activity_Log TSV before any downstream processing occurs. This allows post-hoc inspection of every raw model output to identify hallucinated fields, fabricated patient data, or schema violations that passed the JSON parse check but contain incorrect values.

Important Note: Following Agile principles, testing activities take place iteratively throughout development and are not limited to a single final phase. Test cases are refined and actual results are populated progressively as features reach a testable state.

End of Quality Assurance Testing Documentation

Team Shrooms | Clinical Diagnostics Triage & Workflow Copilot